DATA STRUCTURES & ALGORITHMS

DAY 1 — Expanded English Notes

Based on the uploaded lecture transcript

> **Study sequence:** Understand the idea → trace on paper → write Java code yourself → run and test.

Clickable Index

1. Module overview
2. Why learn DSA?
3. Data structures
4. Abstract Data Type (ADT)
5. Efficiency analysis
6. Space complexity
7. Time complexity
8. Big O, Omega and Theta
9. How to study algorithms
10. Linear search
11. Java implementation
12. Linear-search complexity
13. Quick revision
14. MCQs
15. Practice checklist

These notes follow the uploaded transcript. Extra clarifications are labelled so they are not confused with the lecturer’s simplified wording.

1. Module overview

The lecturer introduced Data Structures and Algorithms (DSA) as a practical module. The stated prerequisite is Java fundamentals: variables, loops, if/else, switch statements and basic OOP concepts. Inheritance and polymorphism are not the main focus.

DSA is language-agnostic: the same underlying data structure or algorithm can be implemented in Java, C++, Python or another language. Syntax changes, but the concept remains.

Topics listed in the lecture

• Time and space complexity

• Linear and binary search

• Recursion

• Selection, bubble and insertion sort

• Linked lists, stacks and queues

• Trees and binary search trees

• Heaps and heap sort

• Quick sort and merge sort

• Hashing and hash tables

• Graph theory and graph implementation

• Divide and conquer, greedy algorithms, dynamic programming and brute force

> Sir mentioned roughly 10 working days for that batch. Dates and evaluation details in the transcript are batch-specific.

2. Why learn DSA if Java has Collections?

Java provides ready-made collections, but Sir explained that DSA is not just about using library classes. The goal is to understand the logic and implementation behind data structures.

• Build logical and algorithmic thinking: turn a problem into clear steps.

• Improve coding: understand an algorithm before translating it into code.

• Prepare for interviews: explain your reasoning rather than reproduce memorised code.

• Understand how lists, stacks, queues and hash tables can work internally.

```
Understand problem -> design algorithm -> dry-run on paper -> code -> run and test
```

The lecturer advised students not to memorise sorting code blindly. Understand the algorithm, trace it manually, and then implement it.

3. Data structures

A data structure is a way of organising data. Sir described two main concerns: how data is arranged in memory, and how operations are performed efficiently.

Structure	Basic idea
Array	Elements stored contiguously; access by index.
Linked list	Nodes are connected to other nodes.
Stack	Processes items in LIFO order.
Queue	Processes items in FIFO order.
Tree / BST	Hierarchical structure; suitably organised trees can support efficient searching.
Heap	Specialised structure often used for priority queues.
Graph	Represents relationships between vertices and edges.

> This is a rough division from the lecture. Built-in support varies by language and library.

4. Abstract Data Type (ADT)

ADT stands for **Abstract Data Type**. It describes what operations a structure provides, without requiring you to focus first on its internal implementation. Sir compared it with using a watch: you care about reading/changing the time, not the internal chip.

Array ADT

• get(index): retrieve an element at an index.

• set(index, value): update the element at an index.

Searching and sorting are higher-level operations performed on arrays.

Stack ADT

• push(value): add to the top.

• pop(): remove the top item.

• peek()/top(): inspect the top without removing it.

• isEmpty(): check whether the stack is empty.

Stack = LIFO (Last In, First Out).

Queue ADT

Queue = FIFO (First In, First Out). The earliest item added is generally the first removed. Common operations include enqueue, dequeue, front/peek and isEmpty.

> An ADT describes expected behaviour; an implementation explains how it is built. A stack can be implemented using an array or linked list.

5. Efficiency analysis

Exact execution time and memory depend on hardware, architecture, data types, implementation and machine configuration. An algorithm taking 10 ms on one machine is not automatically better than one taking 15 ms on a slower machine.

Analysis	Measures	Limitation / use
Exact	Actual bytes, milliseconds, CPU cycles, etc.	Machine-dependent.
Asymptotic	How resource use grows with input size.	Useful for comparing growth as inputs get larger.

Common input-size symbols: n = number of elements, V = number of vertices, E = number of edges, and tree height.

6. Space complexity

Space complexity describes how memory use grows with input size. The lecture separates it into:

• **Input space:** memory needed to represent the input.

• **Auxiliary space:** extra working memory, such as temporary variables, extra data structures or recursive call frames.

Iterative factorial example

```
int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

This iterative version uses a fixed number of extra variables, so auxiliary space is O(1). A recursive factorial creates a chain of function calls; its call stack typically grows with n, so auxiliary space is O(n).

• An n × n matrix stores n^2 entries, so its storage grows as $O(n^2)$.

> In everyday DSA questions, “space complexity” often means auxiliary space. Check whether input storage is included.

7. Time complexity

Time complexity describes how work grows with input size. It is not the exact number of milliseconds. Sir emphasised checking what the loop count actually depends on.

Complexity	Pattern	Example
O(1)	Constant work	Access an array element by index.
O(log n)	Repeatedly reduce problem size	Binary search on sorted data.
O(n)	One pass over n items	Linear search / single input-dependent loop.
O(n log n)	Linear work across logarithmic levels	Merge sort.

Complexity	Pattern	Example
$O(n^2)$	Two input-dependent nested loops	Typical selection/bubble sort.
$O(n^3)$	Three input-dependent nested loops	A triple-loop algorithm such as Floyd–Warshall.

Key lesson from Sir

Do not judge complexity by loop count alone. Ask how many times each loop runs and whether that count changes with the input.

A loop that always runs exactly 10 times is O(1) with respect to n. A loop that runs from 1 to n is O(n).

> Nested loops are often $O(n^2)$ or $O(n^3)$ only when their iteration counts depend on n in the relevant way. A fixed inner loop does not automatically make an algorithm quadratic.

8. Big O, Omega and Theta

Notation	Meaning	Memory aid
Big O — O(f(n))	Asymptotic upper bound.	Upper bound on growth.
Big Omega — Ω(f(n))	Asymptotic lower bound.	Lower bound on growth.
Big Theta — Θ(f(n))	Tight bound: upper and lower bounds.	Same asymptotic order above and below.

> Accuracy note: Big O does not inherently mean worst case, and Omega does not inherently mean best case. They describe bounds. Introductory examples often use O for worst-case and Omega for best-case analysis, but these concepts are not identical.

For linear search, the best case takes constant time while the worst case grows linearly.

9. How to study algorithms

• Keep a notebook and pen/pencil nearby.

• Understand the idea before looking at code.

• Trace the algorithm on paper with a small input.

• Try a similar example independently.

• Then write and run the Java code.

• In the lab, re-type important algorithms instead of relying only on copy-paste.

• Discuss logic and approaches with classmates; do not simply collect a ready-made solution.

Sir recommended more independent practice time for complex algorithms such as quick sort and merge sort than for very simple algorithms.

10. Linear search

Linear search finds a target by checking items one by one, starting from the first. It works even if data is unsorted and can be used with arrays or other sequential structures.

Algorithm

• Start at index 0.

• Compare the current element with the target.

• If equal, return the current index.

• Otherwise move to the next index and repeat.

• If all elements have been checked, report not found.

Index	Value	Target 77?	Action
0	12	No	Continue
1	35	No	Continue
2	77	Yes	Return index 2

The values are an added example; Sir's main point was to compare elements sequentially until a match is found.

> Returning an index is common. Return -1 if the target is absent because index 0 is valid.

11. Linear search in Java

```
public class LinearSearch {
    static int linearSearch(int[] arr, int key) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] numbers = {12, 35, 77, 18, 42};
        int key = 77;
        int index = linearSearch(numbers, key);

        if (index != -1)
            System.out.println("Found at index: " + index);
        else
            System.out.println("Not found");
    }
}
```

• i = 0 starts at the first valid index.

• i < arr.length prevents accessing beyond the last index.

• arr[i] == key compares the current value with the target.

• return i stops at the first match.

• return -1 runs only if no element matches.

> If the array has duplicates, this version returns the first matching index. In {4, 7, 7}, searching for 7 returns index 1.

12. Linear-search complexity

Case	Situation	Complexity
Best	Target is first; one comparison.	O(1)
Worst	Target is last or absent; up to n checks.	O(n)
Average successful search	Roughly (n+1)/2 comparisons if each position is equally likely.	Θ(n)
Auxiliary space	Fixed number of extra variables.	O(1)

n/2 is still linear because constant factors such as 1/2 are ignored in asymptotic growth. If the target is absent, all n elements are checked.

> Input space is separate from auxiliary space. Linear search does not need to create another copy of the input array.

13. Quick revision

Question	Answer
Why learn DSA?	Problem-solving, algorithm design, implementation and interview reasoning.
What is an ADT?	Expected operations/behaviour independent of implementation details.
Stack / queue?	Stack = LIFO; queue = FIFO.
Time complexity?	Growth in work as input size grows.
Space complexity?	Growth in memory use as input size grows.
Auxiliary space?	Extra working memory beyond input representation.
Linear search best/worst?	Best O(1); worst O(n).
Why return -1?	Signals not found; 0 is a valid index.

14. MCQs

1. Why learn DSA if Java has collections?

A. Memorise names

B. Understand logic and problem-solving

C. Avoid coding

D. Replace Java

Answer: B. DSA develops reasoning and implementation skills.

2. DSA is language-agnostic because:

A. Syntax is identical

B. Concepts can be implemented in different languages

C. It needs no code

D. Only Java supports it

Answer: B. Underlying concepts are independent of a specific language.

3. An ADT focuses on:

A. Internal memory only

B. Operations and behaviour

C. Syntax only

D. Screen design

Answer: B. An ADT abstracts the implementation.

4. A stack follows:

A. FIFO

B. LIFO

C. Sorted order

D. Random order

Answer: B. Last In, First Out.

5. A queue follows:

A. LIFO

B. FIFO

C. Largest first

D. Reverse order

Answer: B. First In, First Out.

6. Iterative linear-search auxiliary space is:

A. O(n)

B. $O(n^2)$

C. O(1)

D. O(log n)

Answer: C. It uses a fixed number of extra variables.

7. One loop that runs n times is usually:

A. O(1)

B. O(log n)

C. O(n)

D. $O(n^3)$

Answer: C. Iterations grow with n.

8. A loop that always runs 10 times is:

A. O(n)

B. O(1) with respect to n

C. $O(n^2)$

D. O(log n)

Answer: B. Its work does not grow with n.

9. Linear search worst-case time:

A. O(1)

B. O(log n)

C. O(n)

D. O(n^2)

Answer: C. The target may be last or absent.

10. Why return -1 when not found?

A. Indexes start at -1

B. It is the last index

C. 0 is valid, so -1 signals absence

D. Java requires it

Answer: C. -1 is a conventional sentinel value.

11. Big O describes an asymptotic:

A. Lower bound

B. Upper bound

C. Exact runtime

D. Average case only

Answer: B. Big O is an upper bound.

12. n/2 operations simplify to:

A. O(1)

B. O(n)

C. O(n^2)

D. O(log n)

Answer: B. Constant factors are ignored.

13. Before coding, Sir recommended:

A. Memorise code

B. Trace the algorithm on paper

C. Skip to testing

D. Use collections only

Answer: B. Paper dry-runs build understanding.

14. Linear search requires sorted input:

A. Always

B. Descending order

C. No, it can search unsorted data

D. Only strings

Answer: C. It checks items sequentially.

15. Auxiliary space means:

A. Input only

B. Extra working memory

C. Runtime in milliseconds

D. Output lines

Answer: B. It is additional working memory.

15. Practice checklist

- ■ Explain why DSA matters even when libraries provide collections.
- ■ Distinguish a data structure from an algorithm.
- ■ Explain ADT using a stack or queue.
- ■ Distinguish input space from auxiliary space.
- ■ Explain why exact milliseconds are unreliable across machines.

- ■ Recognise O(1), O(log n), O(n), O(n log n), $O(n^2)$ and $O(n^3)$.
- ■ Explain O, Ω and Θ at a basic level.
- ■ Dry-run linear search and return the first matching index.
- ■ Explain best, worst and average cases for linear search.
- ■ Write the Java linear-search method without copying.

End of Day 1 notes. Revise the definitions, then write linear search from memory and test a target at the beginning, middle, end, and absent from the array.